

Individual Reflection:Fletcher Baccus, Amir Taylor

While developing Subsystem B for the MOSS project, we made several design decisions to keep the simulator simple and easy to understand. The subsystem focuses on virtual memory concepts like paging and page replacement without using real operating system memory management.

One important decision was using a simple 16-bit logical address split into an 8-bit page number and an 8-bit offset. This made address translation easier to follow and debug. We also used small memory sizes like 256-byte pages and only four physical frames so page replacement could be tracked step-by-step. Real operating systems are much larger and more complex, but this setup made the simulator easier to test and learn from.

Another design choice was separating the simulator data from the replacement algorithms. The main simulator structure stores frame data and statistics while FIFO and LRU each handle their own replacement logic. This made the code easier to organize and compare between algorithms. It also allows more algorithms to be added later.

FIFO and LRU were designed differently because they show different ideas. FIFO is simple because it removes the oldest loaded page first. LRU is more advanced because it keeps track of recently used pages using timestamps. LRU requires more work but better demonstrates how operating systems improve memory performance.

Some assumptions were made to keep the project focused. The simulator assumes all addresses are valid and memory access happens in a single-threaded environment. It also only tracks page numbers instead of storing real data in memory. This keeps the focus on address translation and replacement behavior.

The subsystem also has limitations. The number of frames and page sizes cannot change while the program is running. The simulator also does not include advanced features like dirty bits, TLBs, swapping, or multi-process memory spaces. Input validation is basic and may not handle incorrect user input perfectly.

Overall, the project successfully demonstrates the basics of paging, address translation, FIFO, and LRU in a controlled environment. The design focuses more on learning and simplicity than full operating system realism.